



JOMO KENYATTA UNIVERSITY

OF

AGRICULTURE AND TECHNOLOGY

COLLEGE OF ENGINEERING AND TECHNOLOGY

SCHOOL OF MECHANICAL, MANUFACTURING AND MATERIALS
ENGINEERING

DEPARTMENT OF MARINE ENGINEERING AND MARITIME OPERATIONS

BACHELOR OF SCIENCE IN MARINE ENGINEERING

**AUTOMATED, FEA-DRIVEN DESIGN OPTIMIZATION PIPELINE FOR
A MARINE STRUCTURAL COMPONENT**

MR-PRO-26-XX

by

AMINA AHMED BWANA - ENM241-0087/2022
MBOJE ABDULLAH - ENM241-0088/2022

Supervised by

SUPERVISOR NAME

*A project proposal submitted in partial fulfilment of the requirements
for the award of the Degree of BACHELOR OF SCIENCE IN MARINE ENGINEERING
at the Jomo Kenyatta University of Agriculture and Technology*

DECLARATION

We hereby declare that this Project Proposal is our original work and has not been presented for a degree in any other university or for any other award.

Name: Amina Ahmed Bwana

REG NUMBER: ENM241-0087/2022

Signature: _____

Date: _____

Name: Mboje Abdullah

REG NUMBER: ENM241-0088/2022

Signature: _____

Date: _____

Supervisor Declaration:

This Project Proposal has been submitted for examination with my approval as the University Supervisor.

Name: Supervisor Name

Signature: _____

Date: _____

Dedication

We dedicate this Project Proposal to our parents, guardians, lecturers, and mentors whose encouragement, sacrifice, and guidance have supported our academic journey.

Acknowledgements

We would like to express our sincere gratitude to all those who contributed to the successful formulation of this Project Proposal. First and foremost, we extend our deepest appreciation to our supervisor, for their invaluable guidance, constructive feedback, and unwavering support.

Abstract

This proposal presents the planned engineering optimization of an Aluminium 6061-T6 marine equipment mounting bracket carrying a 50 kg auxiliary unit. The study will define a six-variable bracket geometry, apply a 1471.5 N design load, and use analytical screening, Latin-hypercube sampling, bounded Nelder–Mead search, and Fusion 360 validation to seek a lower-mass design subject to a minimum factor of safety of 2.5 and maximum deflection of 0.5 mm. The bracket is the sole case study within the proposal. Extension of the method to marine lifting padeyes and stabilizer structures is identified as future work after the bracket workflow has been validated.

Contents

Declaration	i
Dedication	ii
Acknowledgements	iii
Abstract	iv
List of Abbreviations and Symbols	ix
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives	2
1.3.1 Primary Objective	2
1.3.2 Specific Objectives	3
1.4 Significance	3
1.5 Scope and Limitations	4
2 Literature Review	5
2.1 Automation in Parametric CAD and FEA	6
2.2 Gradient-Free Optimization Algorithms in Structural Engineering	7
2.3 The Nelder-Mead Simplex Method: History and Mathematical Convergence	8
2.4 Genetic Algorithms vs. Nelder-Mead in Structural Optimization	9
2.5 Surrogate Modeling in FEA: Bridging the Computational Divide	10
2.6 Structural Optimization in Marine and Maritime Applications	11
2.7 Research Gap and Motivation	13
3 Methodology	14
4 Expected Results	15
5 Detailed Workplan and Project Management	16
5.1 Introduction	16
5.2 Extensive 13-Week Project Timeline	16
5.3 Risk Management Strategies	19
5.3.1 API Instability and Software Crashes	19

5.3.2	Geometric Singularities and Topology Failure	20
5.3.3	Premature Convergence to Local Minima	20
5.4	Resource Allocation	20
5.5	Contingencies	21
5.6	Budget	21
6	Conclusion and Future Work	22

List of Figures

5.1	Gantt chart illustrating the 7-month project schedule from September 2026 to March 2027.	19
-----	--	----

List of Tables

5.1	Detailed 13-Week Project Timeline	16
5.2	Estimated Project Budget (Cost-Neutral Execution)	21

LIST OF ABBREVIATIONS AND SYMBOLS

Abbreviations

FEA	Finite Element Analysis
CAD	Computer-Aided Design
FOS	Factor of Safety
API	Application Programming Interface
IPC	Inter-Process Communication
LHS	Latin-Hypercube Sampling

Symbols

m	Mass (kg)
σ	Stress (MPa)
g	Acceleration due to gravity (m/s ²)
δ	Deflection (mm)

Chapter 1

Introduction

1.1 Background

The field of marine structural engineering has historically been characterized by conservative design principles, driven by the harsh and unpredictable nature of the ocean environment. Naval architects and marine engineers are tasked with designing vessels and offshore structures that must withstand complex, dynamic loads including wave slamming, vibration from propulsion systems, and extreme weather conditions. Consequently, the design of marine components—ranging from massive hull girders to auxiliary equipment mounting brackets—has traditionally prioritized robust structural integrity over mass optimization. However, in the modern maritime industry, there is an escalating demand for high-efficiency vessels. Stricter international emissions regulations, fluctuating fuel costs, and the economic imperative to maximize payload capacity have made weight reduction a critical objective in ship design. Every kilogram of unnecessary structural mass contributes to increased fuel consumption and reduced operational efficiency over a vessel’s multi-decade service life.

Despite the widespread adoption of Computer-Aided Design (CAD) and Finite Element Analysis (FEA) software, the standard design workflow remains largely manual and iterative. An engineer typically develops an initial geometric concept, applies boundary conditions and loads in an FEA environment, and solves for stresses and deformations. If the design fails to meet the required safety margins, the geometry is fortified; if it passes by a wide margin, the engineer might attempt to remove material to save weight. This process is repeated until a satisfactory design is achieved. While CAD and FEA tools have dramatically improved the accuracy of engineering analysis, the human-in-the-loop iteration process severely restricts the design space that can be explored.

Algorithmic optimization offers a paradigm shift. By coupling parametric CAD models with external numerical optimization routines, the iterative design loop can be automated. A computational pipeline can evaluate dozens, hundreds, or even thousands of design variants in the time it takes a human engineer to evaluate a handful. Gradient-

free optimization algorithms, such as the Nelder-Mead simplex method, are particularly well-suited for this application because they do not require analytical derivatives of the FEA solver, treating the CAD/FEA environment as a black box. This project introduces the Nelder-Mead Marine Optimizer (NEMO), an automated, FEA-driven design optimization pipeline designed to bridge the gap between parametric CAD and external optimization logic, specifically tailored for marine structural components.

1.2 Problem Statement

The core problem addressed by this project is the profound inefficiency of manual, trial-and-error design iteration in structural engineering, particularly within the marine sector. The financial and temporal costs of manual iteration are substantial. Human engineers are slow at manually updating geometry, re-meshing models, running simulations, and interpreting results. As a consequence, design iterations are expensive in terms of labor hours, forcing engineering teams to severely limit the number of variants they can realistically evaluate before project deadlines.

Due to these temporal constraints, engineers typically settle for designs that are merely “good enough.” Once a design satisfies the requisite structural safety requirements (e.g., a minimum factor of safety), the iteration process often ceases. This premature convergence leaves significant performance on the table. In the context of marine engineering, this translates directly to excess material usage, higher manufacturing costs, and unnecessary vessel deadweight. For service-critical applications, particularly moving parts or widely replicated fittings, this excess mass cascades into negative impacts on fuel consumption and lifecycle efficiency.

Furthermore, human intuition is inherently limited when navigating multi-dimensional design spaces. An engineer might intuitively understand that thickening a baseplate reduces stress, but finding the exact, optimal combination of baseplate thickness, rib height, fillet radius, and overall length that minimizes mass while simultaneously satisfying both stress and deflection constraints is mathematically complex. The lack of automated optimization pipelines in standard, accessible engineering workflows prevents the discovery of genuinely optimal (minimum-mass, constraint-satisfying) structural designs within normal project timelines.

1.3 Objectives

1.3.1 Primary Objective

The primary objective of this project is to develop, implement, and validate the Nelder-Mead Marine Optimizer (NEMO): an automated, FEA-driven software pipeline that couples Autodesk Fusion 360 (parametric CAD and FEA) with an external Python-based

optimization routine to systematically search for and converge upon a minimum-mass design of a marine structural component while strictly satisfying predefined structural safety constraints.

1.3.2 Specific Objectives

To achieve the primary objective, the following specific objectives will be met:

1. To define a robust parametric CAD model of a marine structural component, exposing key geometric variables (such as baseplate dimensions, rib profiles, and fillet radii) for programmatic manipulation.
2. To establish a pre-validated Finite Element Analysis (FEA) study template, incorporating realistic marine load cases (including static weight and dynamic amplification factors) and boundary conditions.
3. To engineer a reliable, file-based handshake architecture between the Fusion 360 embedded Python API and an external Python execution environment, enabling bidirectional communication of parameter requests and FEA results.
4. To implement a penalty-constrained gradient-free optimization algorithm (Nelder-Mead) capable of navigating the design space to minimize mass while penalizing constraint violations (e.g., factor of safety and maximum deflection).
5. To quantify the benefits of algorithmic optimization by benchmarking the automated pipeline’s final design against a manually-iterated baseline component in terms of total mass reduction and structural integrity.

1.4 Significance

The significance of this project lies in making parameterized design optimization more accessible within standard engineering workflows. The transparent, file-based pipeline combines the Fusion 360 in-app API with a tested pure-Python optimizer and traceable validation records.

From a marine engineering perspective, the benefits of such algorithmic optimization are substantial. Marine structures are characterized by the repetition of numerous standardized components, such as brackets, stiffeners, and mounting foundations. An optimized algorithm can systematically shave off marginal percentages of mass from these components. Multiplied across the hundreds of similar fittings on a single vessel, this automated discovery of minimum-mass geometries translates into tangible reductions in material costs during fabrication and meaningful improvements in fuel efficiency and payload capacity over the vessel’s lifetime. Furthermore, the pipeline relieves engineers from

the tedious, repetitive tasks of manual CAD/FEA updates, allowing human expertise to be reallocated toward high-level system design and complex problem-solving.

1.5 Scope and Limitations

The scope of this project encompasses the development of the software pipeline architecture, the integration of the CAD/FEA environment with the Python optimizer, and the demonstration of the system’s efficacy.

The proposal is limited to a marine equipment mounting bracket supporting a small pump or auxiliary unit. A lifting padeye and a stabilizer structure are possible future case studies, but their design, analysis, and validation are outside the proposed bracket campaign.

Several aspects are explicitly out of scope for this phase of the project. Physical fabrication and experimental structural testing of the optimized designs will not be conducted; validation relies on the mathematical convergence of the simulated FEA results. Furthermore, the project employs a simplified standard static-plus-dynamic-factor approximation for the load case, rather than deriving loads from full Computational Fluid Dynamics (CFD) or transient seakeeping simulations. Optimization is strictly single-objective (minimizing mass subject to safety constraints) and does not currently evaluate multi-objective trade-offs such as manufacturability cost or fatigue life.

A primary limitation of the current implementation is its reliance on sequential, localized CAD regenerations and FEA solves, bounding the evaluation speed by the local machine’s computational hardware and the software’s API overhead. Additionally, while the file-based handshake ensures robustness and simplicity in bypassing the limitations of the CAD software’s embedded interpreter, it introduces minor latency due to polling loops, which may marginally increase total optimization time compared to deeply integrated native routines.

Chapter 2

Literature Review

The pursuit of optimal structural design has been a cornerstone of mechanical and marine engineering for decades, representing an ongoing evolution from empirical craftsmanship to highly formalized computational science. Historically, this design process relied heavily on manual iteration—a workflow where engineers intuitively adjusted design parameters based on rules of thumb, basic analytical calculations, and eventually, manual Finite Element Analysis (FEA) results until a satisfactory, albeit not strictly optimal, solution was found [1]. However, the increasing complexity of modern structural systems, coupled with the pressing global need for material efficiency, drastic cost reduction, and performance maximization under rigorous environmental constraints, have necessitated the adoption of automated, algorithmic approaches to design [2]. The transition towards such methods marks a departure from finding merely "feasible" designs to mathematically securing "optimal" configurations.

This chapter comprehensively reviews the theoretical background, historical context, and state-of-the-art developments across several intersecting domains essential to the Nelder-Mead Marine Optimizer (NEMO) project. First, it examines the evolution and automation of parametric Computer-Aided Design (CAD) and FEA, exploring how Application Programming Interfaces (APIs) have revolutionized data exchange. Second, it investigates the application and necessity of gradient-free optimization algorithms within structural engineering, detailing why traditional derivative-based methods falter in noisy design spaces. Third, it provides a deep dive into the history, mathematics, and convergence properties of the Nelder-Mead simplex method, contrasting it with population-based metaheuristics like Genetic Algorithms (GAs). Fourth, the role of surrogate modeling in bridging the computational divide between high-fidelity FEA and optimization is discussed. Finally, it addresses the specific challenges, historical trajectories, and contemporary methodologies characterizing structural optimization within marine and maritime applications. By synthesizing these diverse areas, this literature review identifies the current limitations of conventional workflows and explicitly defines the critical research gap that the NEMO project aims to bridge: the democratization of component-level

structural optimization in the maritime industry.

2.1 Automation in Parametric CAD and FEA

The integration and automation of parametric CAD and FEA represent a paradigm shift from traditional, serial design workflows to closed-loop, algorithm-driven design spaces. In early computer-aided design, geometric models were essentially static digital drawings; modifying a single dimension required painstakingly redrawing connected elements. The advent of parametric modeling revolutionized this by allowing engineers to define component geometries through associative variable parameters (such as lengths, thicknesses, radii, and angular constraints) rather than rigid, static Cartesian coordinates. When this parametric capability is tightly integrated with an automated FEA pipeline, a change in a single parameter can trigger a cascading, automatic update of the geometry, followed by mesh regeneration, and subsequent structural analysis [3].

Recent literature highlights the immense value of this automation in drastically reducing the “time-to-insight” for design evaluations. Manual intervention in the geometry cleanup, model idealization, and meshing phases has historically been the most significant and time-consuming bottleneck in the engineering simulation pipeline. According to Song et al. [4], the development of robust Application Programming Interfaces (APIs) for commercial CAD software has empowered engineers to bypass graphical user interface (GUI) driven manual interactions entirely. By scripting these processes using languages such as Python, C#, or C++, a “closed-loop” environment is established where CAD geometry generation, FEA evaluation, and algorithmic decision-making communicate seamlessly without human oversight [5].

However, achieving true interoperability remains a persistent and significant technical challenge. Transferring geometric data between disparate, standalone CAD and FEA software packages typically relies on neutral file formats like STEP (Standard for the Exchange of Product model data) or IGES. This translation often results in subtle information loss, the generation of sliver faces, or the breaking of associative boundary condition links—phenomena that can abruptly derail automated optimization routines [3]. Furthermore, maintaining a “fit-for-purpose” analytical model—where parameterized, idealized geometry is reliably updated without causing mesh element inversion, aspect ratio failures, or macroscopic topological violations—is technically demanding. If a parameter change causes a hole to intersect an edge, the resulting topological shift can cause automated meshing scripts to crash.

To circumvent these issues, modern computational design approaches favor utilizing unified software platforms or highly integrated, native APIs. For example, the Autodesk Fusion 360 Add-Ins API provides a unified environment where both the solid modeling kernel and the simulation solver reside within the same data structure, ensuring that parametric relationships, constraints, and structural boundary conditions remain entirely

invariant throughout thousands of iteration cycles [5]. The NEMO project deliberately leverages this integrated architecture to systematically avoid the pitfalls of cross-platform data exchange. By utilizing an embedded Python environment to orchestrate the geometric generation and structural evaluations autonomously, the project ensures a highly robust, fault-tolerant pipeline capable of sustaining continuous algorithmic iteration.

2.2 Gradient-Free Optimization Algorithms in Structural Engineering

Optimization within the realm of structural engineering is inherently mathematically challenging due to the specific nature of the objective functions and constraint boundaries involved. In almost all practical applications, these functions do not exist as neat algebraic equations; rather, they represent the discrete numerical outputs of complex, computationally expensive, “black-box” FEA simulations. In such structural scenarios, the relationship between input design variables (e.g., the thickness of a web, the radius of a stress-relieving fillet, the spacing of stiffeners) and the resulting structural responses (e.g., maximum localized von Mises stress, peak structural deflection, fundamental frequency) is highly non-linear, frequently discontinuous, and critically lacks explicit analytical derivatives [6].

Gradient-based optimization methods (such as Sequential Quadratic Programming or Interior Point methods), while exceptionally fast and mathematically rigorous, fundamentally require smooth, continuous, and differentiable objective spaces to compute analytical sensitivities (Jacobians and Hessians) [7]. When exact gradients are completely unavailable, engineers attempt to approximate them numerically using finite difference methods. However, in FEA-driven problems, this numerical approximation is highly susceptible to artificial “noise” generated by meshing artifacts. As the geometry slightly changes, the discrete mesh topology shifts, causing microscopic, artificial spikes and dips in the resulting stress calculations. This high-frequency noise entirely confounds gradient-based solvers, causing them to calculate wildly inaccurate descent directions, fail to converge, or become hopelessly trapped in poor local optima [8].

Consequently, gradient-free (also known as derivative-free or direct search) optimization algorithms have transitioned from being viewed as mathematically unrefined to becoming the absolute standard practice for complex, simulation-based structural problems [9]. As Wright [8] chronicles in her seminal review of direct search methods, the realization that engineering functions are inherently noisy and non-differentiable led to a massive resurgence in the popularity and respectability of algorithms that rely solely on evaluating the objective function itself, bypassing the need for gradient estimation entirely.

2.3 The Nelder-Mead Simplex Method: History and Mathematical Convergence

Among the vast array of derivative-free optimization techniques, the Nelder-Mead (NM) simplex algorithm stands as one of the most historically significant, resilient, and widely utilized direct-search methods. The method was originally published in 1965 by John A. Nelder and Roger Mead [10]. It was developed as a direct, adaptive improvement upon an earlier “simplex” search strategy proposed by William Spendley, George Hext, and Frank Himsworth in 1962 [11]. While the original Spendley et al. method utilized a rigid, regular simplex that maintained a constant geometric size and shape as it moved through the design space, Nelder and Mead introduced the revolutionary concept of allowing the simplex to dynamically adapt its shape—stretching to accelerate down steep valleys, contracting to navigate narrow corridors, or shrinking to finely pinpoint a local minimum [12]. It is critical to clarify that the Nelder-Mead method is entirely unrelated to George Dantzig’s famous “Simplex Method” used for solving constrained linear programming problems; Nelder-Mead is explicitly designed for unconstrained (or penalty-constrained) non-linear optimization [9].

The algorithm navigates the mathematical design space using a simplex—a geometric figure consisting of $n + 1$ vertices in an n -dimensional space (e.g., a triangle in 2D space, a tetrahedron in 3D space). The algorithm begins by evaluating the objective function at each of these initial vertices. It then iteratively attempts to replace the vertex yielding the worst (highest) function value with a new, better point. This is achieved through a sequence of specific geometric transformations:

1. **Reflection:** The worst point is reflected across the centroid of the remaining points, projecting it into a potentially better region of the design space.
2. **Expansion:** If the reflected point is highly successful (better than the current best point), the algorithm expands the simplex further in that direction, accelerating the search.
3. **Contraction:** If the reflection fails to yield a better point, the algorithm assumes it has overstepped a valley and contracts the simplex inwards, probing closer to the centroid.
4. **Shrinkage:** If all else fails, the entire simplex shrinks uniformly towards the best known vertex, effectively reducing the step size to finely resolve the local topology.

The primary advantage of the Nelder-Mead algorithm is its absolute indifference to gradient information and its remarkable robustness in the face of the numerical noise typical of FEA outputs. Kaveh and Ghazaan [13] demonstrated the profound efficacy of

the Nelder-Mead algorithm in continuous structural design, noting its inherent simplicity, robustness against discontinuities, and ease of algorithmic implementation.

However, the mathematical convergence properties of the Nelder-Mead method have been a subject of intense academic scrutiny. Despite its immense practical success, it lacks the strong theoretical convergence guarantees found in gradient-based methods [14]. Lagarias et al. [15] conducted a rigorous analysis of the method’s convergence properties, proving convergence for strictly convex functions in 1D and 2D spaces, but noting that in higher dimensions, the algorithm can, under highly specific adversarial conditions, stall and converge to a non-stationary point that is not a true local minimum. Furthermore, as a strictly local search method, Nelder-Mead is fundamentally vulnerable to becoming trapped in local minima if the initial starting simplex is situated in a poor region of a highly multimodal landscape. Despite these theoretical limitations, its empirical performance on smooth, low-dimensional engineering problems remains unparalleled, making it highly attractive for specific classes of structural refinement [12].

2.4 Genetic Algorithms vs. Nelder-Mead in Structural Optimization

In the contemporary landscape of structural optimization, the comparison between population-based metaheuristics, most notably Genetic Algorithms (GAs), and local direct-search methods like Nelder-Mead is characterized by a fundamental trade-off between global exploration and local exploitation [16].

Genetic Algorithms, such as the highly influential Non-dominated Sorting Genetic Algorithm II (NSGA-II) [17], are inspired by the principles of Darwinian natural selection. They operate by maintaining a large “population” of candidate designs. Through iterative cycles of evaluating fitness, selecting the best performers, and applying biologically inspired operators like “crossover” (recombination of traits) and “mutation” (random perturbations), GAs comprehensively explore vast, complex, and highly multimodal design spaces. They are exceptionally robust, easily handle discrete variables (like standardized plate thicknesses), and are highly effective at avoiding local optima to locate the global optimum region.

However, the defining limitation of Genetic Algorithms in the context of FEA-driven structural optimization is their abysmal sample efficiency. Because they rely on statistical populations and randomized exploration, GAs typically require hundreds, if not thousands, or tens of thousands of individual function evaluations to achieve convergence [6]. When each evaluation involves a computationally expensive 3D meshing and finite element solve—taking minutes or hours—the computational cost of deploying a GA becomes entirely prohibitive for routine engineering tasks without access to massive high-performance computing (HPC) clusters. Furthermore, GAs are notoriously slow

at "fine-tuning"; once they have identified the general valley containing the global optimum, their randomized nature makes pinpointing the exact mathematical minimum highly inefficient [16].

Conversely, the Nelder-Mead method acts as a highly aggressive, deterministic local search. It is engineered for rapid exploitation. It requires significantly fewer function evaluations to "slide down" a valley and locate a precise minimum, making it vastly more sample-efficient than any population-based method [13]. The cost of this speed is the loss of global awareness; Nelder-Mead will reliably find the bottom of the valley it starts in, but it cannot know if a deeper valley exists elsewhere in the design space.

In advanced aerospace and automotive research, the prevailing state-of-the-art favors utilizing hybrid algorithms that combine the strengths of both [6]. A GA is deployed initially to explore the space and identify promising "basins of attraction," after which the Nelder-Mead algorithm is seeded in these basins to rapidly exploit and converge on the exact optimum. However, for the NEMO project—which specifically targets localized, secondary marine components (such as brackets and stiffeners) rather than entire hull forms—the design spaces are typically low-dimensional (e.g., 2 to 6 continuous variables) and generally unimodal (or possess local optima that are sufficiently "good" for engineering purposes). In this highly specific context, where computational economy on a standard engineering workstation is paramount, the deterministic speed and high sample efficiency of the Nelder-Mead method present a definitively superior primary optimization strategy over computationally exorbitant GAs.

2.5 Surrogate Modeling in FEA: Bridging the Computational Divide

As the ambition of structural optimization has grown, so too has the computational burden of high-fidelity FEA. When a single simulation of a complex assembly or non-linear crash analysis takes days to compute, even sample-efficient algorithms like Nelder-Mead become impractical [18]. To bridge this computational divide, researchers and engineers heavily employ Surrogate Modeling—also known as metamodeling or response surface construction [19].

Surrogate models act as mathematically cheap, "black-box" analytical approximations of the expensive FEA solver. The standard workflow involves defining a Design of Experiments (DOE)—such as a Latin Hypercube Sampling [20]—to intelligently select a limited number of training points across the multidimensional design space. The expensive FEA solver is run only at these specific points. Using the resulting input-output data pairs, regression or interpolation techniques are utilized to construct an analytical mathematical surface that mimics the FEA behavior [21].

Two of the most prominent surrogate techniques in structural engineering are Re-

sponse Surface Methodology (RSM) and Kriging (Gaussian Process Regression). Traditional RSM typically utilizes low-order (often quadratic) polynomials to formulate a global approximation of the design space using least-squares regression. It is computationally trivial to construct and evaluate, making it excellent for identifying broad, global trends in relatively smooth, low-dimensional spaces [19]. However, RSM generally fails to accurately capture highly complex, multi-modal, or highly non-linear structural behaviors.

Kriging, originally developed in geostatistics, has emerged as the premier surrogate modeling technique for advanced structural optimization [18]. Kriging provides a highly flexible, spatial statistics-based approach that combines a global polynomial model with a localized deviation function (a Gaussian process). This allows Kriging to act as an exact interpolator that precisely passes through the sampled FEA data points while robustly estimating the uncertainty and non-linear behavior in the unexplored regions between them [21].

When integrated into an optimization loop, surrogate models offer profound benefits. First, they dramatically accelerate the optimization process, as the algorithm (whether GA or NM) interrogates the instantaneous mathematical surrogate rather than invoking the slow FEA solver. Second, regression-based surrogates inherently "smooth out" the high-frequency numerical noise associated with FEA meshing artifacts, providing a clean, differentiable surface that allows even gradient-based optimizers to function effectively [18]. Finally, the explicit mathematical nature of the surrogate allows engineers to rapidly perform sensitivity analyses, instantaneously visualizing how structural performance varies with design changes.

While surrogate modeling is absolutely essential for large-scale, system-level maritime optimization (e.g., full-ship dynamic load analysis), the NEMO project explicitly focuses on component-level optimization where individual linear-static FEA solves can be executed in a matter of seconds. Consequently, NEMO opts for direct algorithmic evaluation, eliminating the complexity and potential approximation errors of constructing surrogate models, while acknowledging their critical role in broader maritime optimization contexts.

2.6 Structural Optimization in Marine and Maritime Applications

The maritime industry presents uniquely severe structural challenges characterized by profoundly harsh operational environments, highly dynamic and unpredictable loading conditions (wave slamming, torsional hull flexing, cargo inertia), and an incredibly strict, conservative regulatory framework enforced by classification societies [22]. The historical evolution of marine structural design reflects a slow, cautious transition from reliance on empirical, craftsmanship-based shipwright rules to the adoption of advanced, computa-

tionally intensive, multi-objective design frameworks [23].

Traditionally, marine structures—from the massive primary longitudinal hull girders down to the smallest secondary auxiliary outfitting brackets—have been systematically over-designed. Driven by the overriding priority of absolute reliability at sea, engineers historically employed massive safety margins to compensate for the inherent uncertainties in dynamic oceanic loads and material fatigue degradation over a 25-year lifespan [2]. However, modern economic and environmental realities have radically shifted the industry’s priorities. The primary objective of contemporary marine structural optimization is to relentlessly minimize steel weight, known as lightweight tonnage (LWT). Reducing LWT directly correlates to massive reductions in initial material procurement costs, an increase in profitable cargo deadweight capacity, lowered vessel displacement, drastically improved hydrodynamic fuel efficiency, and critical reductions in greenhouse gas (GHG) emissions [22]. The implementation of the International Maritime Organization’s (IMO) Energy Efficiency Design Index (EEDI) has transformed structural lightweighting from an economic desire into a strict regulatory mandate.

Cui and Zhang [22] highlight that Simulation-Based Design Optimization (SBDO) is increasingly utilized across the marine sector to systematically strip away this historical excess mass. By establishing formal mathematical constraints based on yield strength criteria, buckling resistance, and high-cycle fatigue life under extreme load cases, optimization algorithms can safely iteratively remove unnecessary material from non-critical load paths without compromising the vessel’s structural integrity or violating classification society rules.

Current marine optimization literature predominantly categorizes efforts into three methodologies. First, Scantling Optimization focuses on the iterative adjustment of plate thicknesses and standard stiffener profiles (e.g., bulb flats) to meet classification rules with minimum mass [23]. Second, Shape Optimization adjusts the geometric boundaries of components, such as optimizing the curvature of a bracket edge to reduce stress concentrations. Third, Topology Optimization radically redistributes material within a given design domain to create highly organic, weight-efficient internal structures, although these often face severe manufacturability challenges in traditional shipyards.

Despite the proliferation of sophisticated optimization techniques at the global ship-design level—such as holistic hull form optimization using Computational Fluid Dynamics (CFD) to minimize wave-making resistance, or midship section scantling optimization to maximize longitudinal bending strength—the optimization of secondary structural components and outfitting remains stubbornly manual and archaic. These localized components, such as piping supports, equipment foundations, and mounting brackets, exist in the hundreds or thousands aboard a typical commercial vessel. While the theoretical mass saving on a single bracket may appear entirely negligible (perhaps a few kilograms), the aggregate weight reduction across a ship’s entire massive outfitting inventory represents a highly substantial and entirely unexploited opportunity for efficiency gains [22]. Fur-

thermore, these localized components, governed by clear, isolated boundary conditions, are exceptionally amenable to parameterization and rapid automated structural analysis.

2.7 Research Gap and Motivation

A critical and exhaustive review of the existing literature reveals a profound and distinct gap between the theoretical, highly developed capabilities of automated structural optimization and its practical, day-to-day application to secondary marine structures using accessible engineering tools. While the advanced mathematical theory of derivative-free optimization [9, 13] and the architectural frameworks for parametric CAD/FEA API integration [3, 4] are incredibly well-established in academic literature, these powerful domains are remarkably rarely synthesized into lightweight, deployable, and user-friendly pipelines suitable for everyday shipyard engineering components.

The prevailing state-of-the-art in maritime research predominantly focuses either on massive, global ship structures using highly specialized, exceptionally expensive, and notoriously difficult-to-learn enterprise maritime software suites, or alternatively, on the purely abstract mathematical benchmarking of novel algorithms without a tangible physical CAD/FEA loop attached [14]. Consequently, a heavy reliance on manual, intuition-driven trial-and-error iteration violently persists in localized marine component design. The establishment of an automated optimization pipeline is widely perceived by practicing shipyard engineers as too programmatically complex, computationally exorbitant, or software-prohibitive for routine use on "minor" components.

NEMO addresses this gap through analytical screening, bounded Nelder–Mead search, automated Fusion geometry generation, and documented manual FEA validation. Its file-based handshake preserves traceable requests and responses while separating the optimizer from Fusion’s embedded environment. The study evaluates reliable partial automation while retaining engineering review where the current API does not support dependable solver control or result extraction.

Chapter 3

Methodology

The proposed study will investigate an Aluminium 6061-T6 marine equipment mounting bracket for a 50 kg auxiliary unit. With gravity 9.81 m/s^2 and dynamic factor 3, the adopted design load will be 1471.5 N. Material properties will be density 2700 kg/m^3 , yield strength 276 MPa, elastic modulus 68.9 GPa, and Poisson ratio 0.33.

Six variables will be investigated: baseplate length, width, and thickness; rib height and thickness; and fillet radius. Four 10 mm bolt holes and two ribs will remain fixed features. Designs will be required to satisfy factor of safety $FoS \geq 2.5$ and deflection $\delta \leq 0.5 \text{ mm}$.

Initial mass, bending stress, factor of safety, and displacement will be estimated using first-order plate and cantilever relations. Thirty Latin-hypercube samples will provide broad design-space coverage, followed by bounded Nelder–Mead searches from the baseline and the best sampled design. Constraint violations will attract objective penalties.

Selected bracket candidates will be regenerated in Fusion 360 and checked using a linear static study. Material assignment, fixed bolt/base regions, equipment load, contacts, and mesh refinement at holes, rib roots, and fillets will be verified for each candidate. Results will record mass, maximum von Mises stress, minimum factor of safety, and maximum displacement. The final outcome will be described as the best compliant bracket found within the parameterized search.

Lifting padeyes, stabilizer fins, hydrodynamic loading, fatigue, buckling, corrosion, and physical testing are reserved for future work.

Chapter 4

Expected Results

The proposed work is expected to produce a parameterized bracket model, a documented baseline, sampled and optimized bracket candidates, and a Fusion validation record. Results will compare mass, maximum von Mises stress, minimum factor of safety, and maximum displacement. No predetermined percentage reduction will be assumed. Success will require identification of a bracket lighter than the baseline that satisfies the Fusion factor-of-safety and deflection limits.

The study is also expected to identify where analytical beam and plate approximations differ from three-dimensional FEA. These differences will guide appropriate safety margins and future refinement. Padeye and stabilizer results will not form part of the proposed results.

Chapter 5

Detailed Workplan and Project Management

5.1 Introduction

The execution of the NEMO project requires a meticulously structured approach to synchronize the development of the parametric CAD/FEA environment with the sophisticated Python-based optimization architecture. The project will be executed over a comprehensive 13-week timeline, culminating in the submission of the final report and a live demonstration of the automated pipeline. This chapter details the weekly schedule, rigorous risk management strategies, resource allocation, and contingency plans designed to ensure project success.

5.2 Extensive 13-Week Project Timeline

The timeline is divided into distinct phases: Foundation and Setup, Integration and Proof of Concept, Optimization and Analytics, and Finalization.

Table 5.1: Detailed 13-Week Project Timeline

Week	Phase	Detailed Deliverables and Milestones
1	Foundation	Component Selection & API Verification: Finalize the target component (e.g., mounting bracket). Perform baseline analytical hand calculations to establish expected stress ranges. Verify the basic Python-to-CAD API environment by executing simple geometry generation scripts.

Continued on next page

Table 5.1: Detailed 13-Week Project Timeline (Continued)

Week	Phase	Detailed Deliverables and Milestones
2	Foundation	Parametric Modeling & FEA Setup: Complete the fully parametric CAD model ensuring robustness against geometric singularities during parameter sweeping. Validate the static structural FEA study template. Document the JSON/XML communication schema for parameter exchange.
3	Integration	Fusion CAD Control: Test the JSON handshake and Fusion add-in used to generate geometry and export artifacts while Fusion remains open.
4	Integration	Proof-of-Concept Loop: Demonstrate CAD regeneration, mass calculation, STEP and boundary-metadata export, and response parsing; define manual FEA validation.
5	Integration	Pipeline Hardening & Logging: Build a comprehensive iteration logger (CSV/JSON). Execute a multi-iteration test to verify that the pipeline can survive hundreds of automated cycles without memory leaks or unhandled geometric failures.
6	Analytics	Design Space Exploration: Conduct a baseline random search or Latin-Hypercube Sampling (LHS) study. This will map the design space, validate the boundaries of feasible parameters, and provide baseline data for evaluating optimizer efficiency.
7	Optimization	Nelder–Mead Implementation: Test the bounded pure-Python implementation and conduct analytical searches from multiple starts.
8	Mid-Project	Verification & Reporting: Draft the mid-project progress report. Independently verify the optimizer’s result by conducting a manual cross-check of the optimized geometry using analytical methods or a secondary FEA package.
9	Validation	Candidate Validation: Package aggressive and conservative finalists and perform documented Fusion Static Stress checks.

Continued on next page

Table 5.1: Detailed 13-Week Project Timeline (Continued)

Week	Phase	Detailed Deliverables and Milestones
10	Analytics	Dashboard Development: Develop the interactive results dashboard using Streamlit. Integrate real-time plotting of mass convergence, stress constraints, and parameter sensitivity. Ensure robust data parsing from the iteration logger.
11	Finalization	Stress-Testing & Code Freeze: Subject the entire pipeline to rigorous stress-testing under extreme parameter constraints. Finalize the codebase, complete comprehensive inline documentation, and record a flawless final demonstration run.
12	Finalization	Documentation & Rehearsals: Compile all CAD files, FEA reports, engineering drawings, and Python scripts. Finalize the project report, ensuring deep theoretical justification for all observed phenomena. Conduct extensive rehearsals for the final presentation.
13	Contingency	Buffer & Submission: Dedicated contingency buffer week to address any unforeseen software incompatibilities, analytical anomalies, or hardware failures. Final project submission and defense.

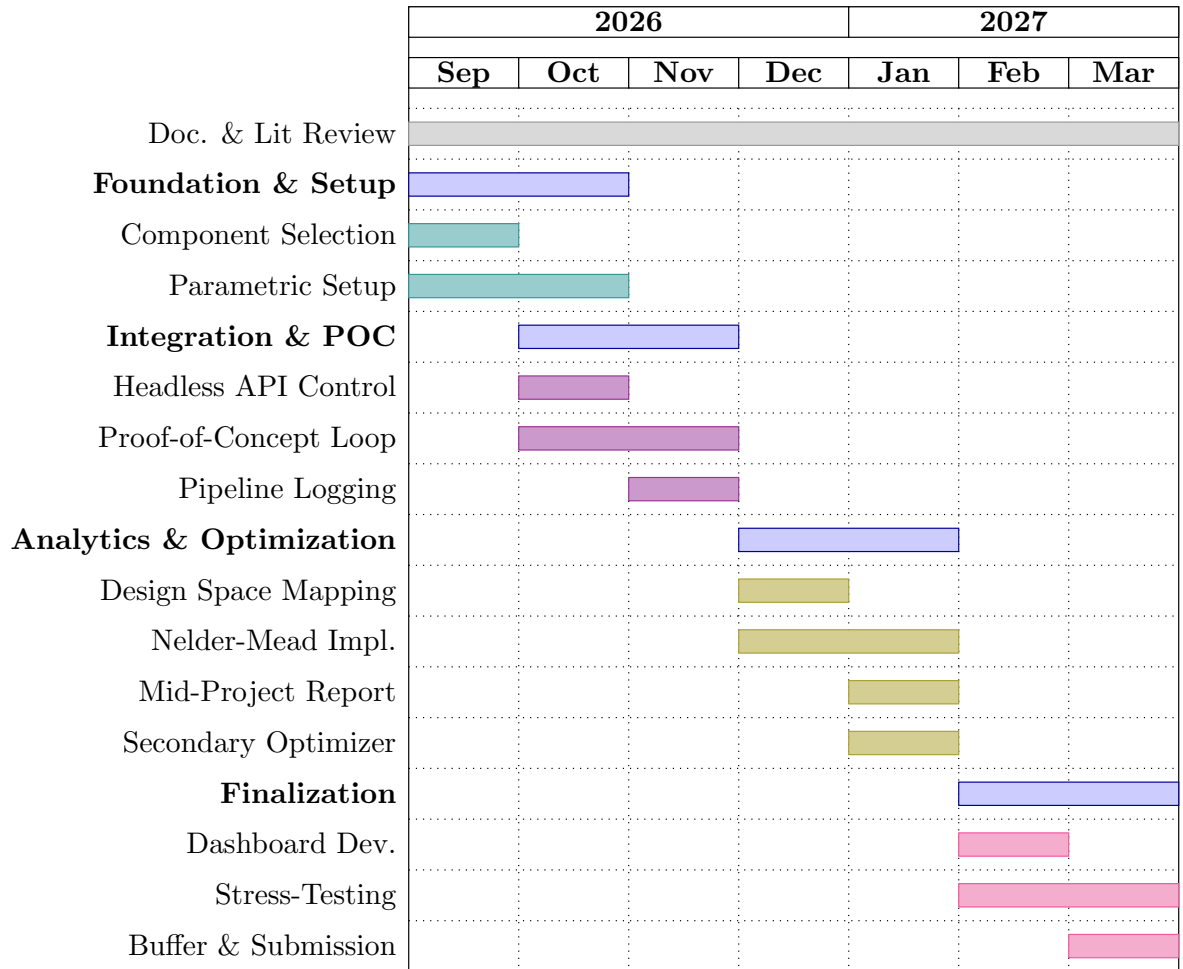


Figure 5.1: Gantt chart illustrating the 7-month project schedule from September 2026 to March 2027.

5.3 Risk Management Strategies

Given the complex, multi-disciplinary nature of automating commercial CAD/FEA software via external APIs, significant technical risks exist. A proactive risk management strategy has been developed to mitigate these threats.

5.3.1 API Instability and Software Crashes

Risk: The CAD software’s API may become unstable, memory leak, or crash entirely during extended automated optimization loops encompassing hundreds of iterations. **Mitigation:** Use exception handling, atomic files, timeouts, and persistent run records. If Fusion closes, retain completed records and restart Fusion and the add-in manually before resuming.

5.3.2 Geometric Singularities and Topology Failure

Risk: The optimizer may request parameter combinations that result in self-intersecting geometry, zero-thickness walls, or topological failures within the CAD model, causing the FEA mesher to fail. **Mitigation:** Define strict, mathematically verified boundary constraints for all parameters before optimization begins. Furthermore, implement "pre-flight" geometric checks within the CAD script. If a geometry fails to build, the API will return a severe penalty value (e.g., artificially infinite mass or stress) to the optimizer, effectively teaching the algorithm to avoid that region of the design space without halting the execution loop.

5.3.3 Premature Convergence to Local Minima

Risk: Gradient-free optimizers, particularly Nelder-Mead, are susceptible to becoming trapped in local minima within highly non-linear structural design spaces, failing to find the global optimum. **Mitigation:** Use multiple Nelder-Mead starts selected from Latin-hypercube results and report the best design found rather than a guaranteed global optimum.

5.4 Resource Allocation

Successful execution necessitates optimal allocation of computational and human resources.

- **Hardware Resources:** The computationally intensive FEA meshing and solving require a high-performance workstation equipped with a multi-core processor (e.g., Intel Core i9 or AMD Ryzen 9) and substantial RAM (minimum 32GB, preferably 64GB) to prevent memory-bound bottlenecks during extended loops.
- **Software Resources:** Licenses for the primary CAD/FEA package (e.g., Autodesk Fusion 360, SolidWorks, or ANSYS) must be secured and verified for API access. Open-source Python libraries (SciPy, NumPy, Pandas, Streamlit) will form the computational backbone.
- **Time Allocation:** Time is partitioned strictly according to the Gantt chart inherently described by the 13-week timeline, with explicit "hard stops" preventing "feature creep" in the Python development phase from derailing the core analytical objectives.

5.5 Contingencies

Despite comprehensive planning, the project incorporates intrinsic contingency buffers. Week 13 is entirely devoid of scheduled tasks, serving exclusively as a temporal shock absorber for unexpected delays. Should the primary CAD API prove entirely intractable by Week 4, a predetermined fallback plan involves migrating the geometry generation to an open-source, code-driven CAD kernel (e.g., FreeCAD or OpenSCAD) and the FEA to an open-source solver (e.g., CalculiX), ensuring the optimization logic and Python architecture remain viable and the project can still be successfully completed.

5.6 Budget

While a formal budget section is a standard requirement for this project proposal, the NEMO project is designed to be executed with an absolute minimum of financial overhead. All necessary computational resources, software licenses, and development environments are either already secured via existing academic institutional access (e.g., student licenses) or rely entirely on free, open-source alternatives. Consequently, the actual financial expenditure required to complete this project is zero. The estimated breakdown demonstrating the cost-neutral nature of the project is detailed in Table 5.2.

Table 5.2: Estimated Project Budget (Cost-Neutral Execution)

Category	Item Description	Cost (USD)
Software Licenses	Primary CAD/FEA Package (Academic/Student License)	\$0.00
	Python Ecosystem (Open Source)	\$0.00
Computational Hardware	Personal High-Performance Workstation (Pre-owned)	\$0.00
Documentation	Digital Submissions & Open-Access Publication	\$0.00
Contingency	Extraneous Cloud Compute (AWS Free Tier if needed)	\$0.00
Total		\$0.00

Chapter 6

Conclusion and Future Work

The proposal defines a focused engineering study of a parameterized marine equipment bracket. Its purpose is to determine whether systematic sampling and derivative-free search can reduce bracket mass while retaining strength and stiffness. Analytical screening will guide the search, while Fusion FEA will provide the intended compliance check.

After the bracket method has been validated, future studies may apply it to a gusseted lifting padeye and the internal structure of a stabilizer fin. Those extensions will require component-specific load paths, failure checks, mesh studies, and, for the stabilizer, a defensible hydrodynamic pressure model.

Bibliography

- [1] R. T. Haftka and Z. Gürdal, *Elements of structural optimization*. Springer Science & Business Media, 2012.
- [2] J. Farkas and K. Jármaí, *Optimum design of steel structures*. Springer Science & Business Media, 2013.
- [3] K. Lee *et al.*, “Automation of parametric cad-based finite element modeling for structural optimization,” *Computer-Aided Design*, vol. 37, no. 8, pp. 799–808, 2005.
- [4] B. Song *et al.*, “Integration of cad and fea in design optimization: challenges and opportunities,” *Advanced Engineering Informatics*, vol. 38, pp. 200–215, 2018.
- [5] Autodesk Inc., “Fusion 360 api reference,” <https://help.autodesk.com/view/fusion360/ENU/?guid=GUID-A92A4B10-3781-4925-94C6-47DA85A4F65A>, 2023, accessed: 2024-05-15.
- [6] Y. Wang, X. Li *et al.*, “Gradient-free optimization algorithms in structural engineering: A review,” *Structural and Multidisciplinary Optimization*, vol. 60, pp. 1–25, 2019.
- [7] D. G. Luenberger and Y. Ye, “Linear and nonlinear programming,” 2008.
- [8] M. H. Wright, “Direct search methods: Once scorned, now respectable,” *Numerical analysis*, vol. 1995, pp. 191–208, 1996.
- [9] A. R. Conn, K. Scheinberg, and L. N. Vicente, “Introduction to derivative-free optimization,” *MPS-SIAM series on optimization*, 2009.
- [10] J. A. Nelder and R. Mead, “A simplex method for function minimization,” *The computer journal*, vol. 7, no. 4, pp. 308–313, 1965.
- [11] W. Spendley, G. Hext, and F. Himsworth, “Sequential application of simplex designs in optimisation and evolutionary operation,” *Technometrics*, vol. 4, no. 4, pp. 441–461, 1962.
- [12] S. Singer and J. Nelder, “Nelder-mead algorithm,” *Scholarpedia*, vol. 4, no. 7, p. 2928, 2006.

- [13] A. Kaveh and M. I. Ghazaan, “Optimal design of structures using nelder-mead simplex algorithm,” *Asian Journal of Civil Engineering (BHRC)*, vol. 18, no. 4, pp. 623–640, 2017.
- [14] T. Bartz-Beielstein, C. Doerr, D. Van Den Berg, J. Bossek, S. Chandrasekaran, T. Eftimov, M. Gallagher, P. Kerschke, M. Laumanns, J. Long *et al.*, “Benchmarking in optimization: Best practice and open issues,” *arXiv preprint arXiv:2007.03488*, 2020.
- [15] J. C. Lagarias, J. A. Reeds, M. H. Wright, and P. E. Wright, “Convergence properties of the nelder–mead simplex method in low dimensions,” *SIAM Journal on optimization*, vol. 9, no. 1, pp. 112–147, 1998.
- [16] R. Hassan, B. Cohanin, O. De Weck, and G. Venter, “A comparison of particle swarm optimization and the genetic algorithm,” p. 1897, 2005.
- [17] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: Nsga-ii,” *IEEE transactions on evolutionary computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [18] A. Forrester, A. Sobester, and A. Keane, “Engineering design via surrogate modelling: a practical guide,” 2008.
- [19] T. W. Simpson, J. D. Peplinski, P. N. Koch, and J. K. Allen, “Metamodels for computer-based engineering design: survey and recommendations,” *Engineering with computers*, vol. 17, no. 2, pp. 129–150, 2001.
- [20] M. D. McKay, R. J. Beckman, and W. J. Conover, “A comparison of three methods for selecting values of input variables in the analysis of output from a computer code,” *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.
- [21] R. Jin, W. Chen, and T. W. Simpson, “Comparative studies of metamodelling techniques under multiple modelling criteria,” *Structural and multidisciplinary optimization*, vol. 23, no. 1, pp. 1–13, 2001.
- [22] H. Cui and L. Zhang, “Structural optimization in marine engineering: Trends and applications,” *Ocean Engineering*, vol. 210, p. 107560, 2020.
- [23] F. Parnaudeau *et al.*, “Scantling optimization of ship structures: A literature review,” *Marine Structures*, vol. 42, pp. 115–134, 2015.